AI Knowledge Base Evaluation Platform: Development Playbook

1. Project Foundation and Strategic Objective

The Core Mission

The objective of this project is to architect a production-grade AI system that meets the rigorous engineering standards of the 2026 job market. We are moving beyond the saturated market of "simple chatbots" to build a sophisticated evaluation platform. The mission is to provide users with a system that not only answers questions from uploaded documents but systematically measures and optimizes the accuracy, reliability, and performance of those answers. This mimics the high-stakes workflows used by professional AI teams to maintain enterprise-level RAG (Retrieval-Augmented Generation) systems.

Contrast: Production vs. Portfolio

To distinguish your work for senior-level recruiters, this project avoids the "Upload-Chat-Hope" anti-pattern in favor of a data-driven engineering cycle.

- **Standard Portfolio Project (Entry-Level):** Document Upload → Chat Interface → AI Answer. (Relies on subjective "vibe checks.")
- **Production-Grade Platform (Senior-Level):** Document Upload → Structured Retrieval → AI Answer → Automated Evaluation → Performance Metrics → Iterative Experimentation. (Relies on objective data.)

The Technology Stack

Every tool in this stack is chosen to signal professional-grade technical judgment.| Tool | Reason for Selection || ------ | ------ || **Next.js** | The 2026 industry standard for AI-native web applications; offers optimal deployment and server-side rendering for LLM interactions. || **Shadcn UI** | Delivers a modern, enterprise-ready aesthetic that signals high-quality front-end standards to recruiters. || **Tailwind CSS** | Facilitates rapid, maintainable styling in a production environment. || **Supabase (Postgres)** | Provides managed PostgreSQL with native pgvector support, authentication, and file storage in a single integrated ecosystem. || **OpenAI (GPT-4.1/5)** | Selected for superior reasoning capabilities and industry dominance; essential for complex RAG tasks. || **OpenAI Embeddings** | Specifically text-embedding-3-small for its optimal balance of cost-efficiency and performance for MVP retrieval. || **LlamaIndex** | Chosen over LangChain because it is built specifically for RAG, offering cleaner abstractions for document indexing and retrieval logic. || **Ragas** | The industry-recognized framework for quantifying RAG performance through specific evaluation metrics. || **Langfuse** | Provides essential observability and tracing; demonstrating proficiency here proves you can debug and audit AI chains. |

2. High-Level System Architecture

The architecture is divided into two distinct paths: the **Inference Path** (executing the user request) and the **Evaluation Path** (measuring the quality of the execution).

The Inference Path (Stages 1–8)

1. **User Upload:** Raw PDF ingestion via the UI.

2. **Document Parser:** Converting PDF blobs into clean, manageable text.
3. **Chunking:** Strategically segmenting text into logical units.
4. **Embeddings:** Mapping text chunks into vector space via text-embedding-3-small.
5. **Supabase Vector DB:** Storing vectors with their associated text metadata.
6. **Retriever:** Executing a similarity search to find context relevant to a user query.
7. **OpenAI Model:** Processing the retrieved context and user question.
8. **Answer Generation:** Delivering a grounded response to the user.

The Evaluation Path (Stages 9–11)

1. **Evaluation Engine:** Running the Ragas framework against the (Context + Question + Answer) triplet.
2. **Metrics Dashboard:** Aggregating evaluation scores for visualization.
3. **Tracing (Langfuse):** Capturing the full lifecycle of the request for observability and auditing.

3. Phase 1: Project Setup and Skeleton

Objective and Rationale

We are establishing the application skeleton first to mitigate deployment risks before we introduce the latency and complexity of LLM integrations. A stable, deployed environment is the prerequisite for professional engineering.

Task Checklist

1. Initialize a Next.js project with TypeScript.
2. Configure Tailwind CSS and install Shadcn UI.
3. Set up a Supabase project and enable the pgvector extension.
4. Connect the Next.js app to Supabase via environment variables.
5. Implement Supabase Auth for secure user access.
6. Build a basic sidebar/dashboard layout.
7. Deploy to Vercel to ensure CI/CD parity.**Architect's Tip:** Never wait until the end of a project to deploy. Production parity (matching your dev environment to your live environment) should be achieved on Day 1.

AI Builder Prompt

"Generate a professional Next.js 14+ dashboard application using Tailwind CSS and Shadcn UI. Implement a sidebar navigation and a secure authentication flow using Supabase Auth (Login/Signup). Create a 'Documents' page and a 'Settings' page. Ensure the layout is responsive and follows enterprise design principles. Provide the project structure and the necessary lib/supabase client configuration."

4. Phase 2: Secure File Management

Objective and Rationale

A RAG system is only as good as its data. We must build a secure pipeline for handling PDF uploads and preserving the metadata required for later retrieval.

Implementation Steps

1. Create a dashboard upload page using Shadcn's Card and Button components.
2. Integrate the Supabase Storage API for file hosting.
3. Create a documents table in PostgreSQL to track file metadata (URL, UserID, Title, FileSize).**Architect's Tip:** Always store the original file in object storage and only keep the reference/metadata in your database. This ensures your system can scale as document volume increases.

AI Builder Prompt

"Build a PDF upload component using Shadcn UI. The component should support drag-and-drop and show a progress bar. On upload, the file must be stored in a Supabase Storage bucket named 'documents'. Simultaneously, create a record in a Supabase PostgreSQL table named 'document_metadata' containing the file name, the public URL, and the user's ID. Include error handling for non-PDF files and upload failures."

5. Phase 3 & 4: Document Processing and Vectorization

The RAG Pipeline

In this phase, we convert static PDFs into dynamic "knowledge chunks." We extract the text, split it logically, and transform it into numerical vectors that enable semantic search.

Technical Requirements

- **Text Extraction:** Clean extraction from PDF headers and body.
- **Strategic Chunking:** Using a fixed-size chunker (initially 500 characters).
- **Vectorization:** Calling OpenAI's text-embedding-3-small.
- **Storage:** Inserting chunks and vectors into a Supabase table with a vector column type.**Architect's Tip:** Chunking is the "silent killer" of RAG systems. If chunks are too small, they lose context; if too large, they introduce noise. This is why our Experimentation Engine (Phase 8) is critical.

AI Builder Prompt

"Create a Next.js API route that handles document processing. The route should: 1. Extract text from a PDF stored in Supabase Storage. 2. Use LlamaIndex's SentenceSplitter to break the text into 500-character chunks. 3. Call the OpenAI text-embedding-3-small model to generate embeddings for each chunk. 4. Save the text chunks and their embeddings into a Supabase table called 'document_chunks' using the pgvector extension."

6. Phase 5 & 6: Retrieval and AI Response Generation

Core Functionality

When a user asks a question, we trigger a semantic search. We compare the vector of the user's question to our stored chunk vectors to find the most mathematically similar context.

Prompt Engineering

We use a strict prompt template to instruct GPT-4.1/5 to act as a grounded assistant. This prevents hallucinations by forcing the model to rely solely on the provided context.**Architect's Tip:** LlamaIndex is chosen here because its VectorStoreIndex

abstraction is significantly cleaner for managing retrieval logic than manual LangChain chains, reducing the potential for "spaghetti code."

AI Builder Prompt

"Using LlamaIndex, write a backend retrieval service. It should take a user query, embed it using OpenAI, and perform a similarity search against the 'document_chunks' table in Supabase. Use the retrieved context to fill a prompt template: 'You are a technical assistant. Answer the user question ONLY using the following context: {context}'. Send the final prompt to OpenAI (GPT-4.1 or GPT-5) and return the generated answer."

7. Phase 7: The Evaluation Framework (The Core Differentiator)

Defining Quality Metrics

This phase separates engineers from hobbyists. We use Ragas to quantify performance:

- **Faithfulness:** Does the answer stay true to the context? (Hallucination check).
- **Answer Relevance:** Does the answer actually address the user's prompt?
- **Context Precision:** Did the retriever find the *exact* chunks needed?
- **Context Recall:** Did the retriever find *all* relevant information from the source?

Automated Testing Workflow

1. Generate a "Ground Truth" test set of question-answer pairs.
2. Run the RAG pipeline for these questions.
3. Pass the (Question, Context, Answer, Ground Truth) to Ragas.
4. Store these scores in an evaluations table for performance tracking over time.

AI Builder Prompt

"Develop an automated evaluation script using the Ragas framework. After the RAG pipeline generates an answer, the script should calculate Faithfulness, Answer Relevance, Context Precision, and Context Recall. Store these metrics in a Supabase table called 'evaluation_logs', linked to the specific question and answer pair. Ensure the script handles the asynchronous nature of LLM calls."

8. Phase 8: The Experimentation Engine

Variable Testing

To optimize the system, we test the trade-off between retrieval precision and context costs. We will vary:

- **Chunk Sizes:** 200 (precise but fragmented), 500 (balanced), 1000 (context-rich but noisy).
- **Retrieval Methods:** Top-k vs. reranking.
- **Prompt Variations:** Chain-of-thought vs. direct answering.

The Leaderboard Concept

The deliverable is a configuration leaderboard. This proves to recruiters that your system's performance isn't a fluke—it’s the result of systematic optimization.

AI Builder Prompt

"Build a comparison dashboard using Shadcn UI and Recharts. The dashboard should display a 'Leaderboard' that compares different RAG configurations (e.g., 'Chunk Size 200' vs 'Chunk Size 1000'). Visualize the Ragas scores (Faithfulness, Relevance, etc.) for each configuration so the user can identify the top-performing setup at a glance."

9. Phase 9: Observability and Tracing

Visibility Requirements

Recruiters in 2026 look for "traceability" as a sign of seniority. We will use Langfuse to record:

- **The Chain:** Exactly which chunks were retrieved for which question.
- **The Latency:** How long each step (embedding, retrieval, generation) took.
- **The Cost:** Token usage for each OpenAI call.

AI Builder Prompt

"Integrate Langfuse tracing into the existing LlamaIndex RAG pipeline. Create a wrapper for the OpenAI and Supabase calls that captures the input prompt, the retrieved context chunks, the similarity scores, and the final LLM output. Ensure all traces are tagged with a 'session_id' for easy debugging in the Langfuse dashboard."

10. Phase 10: Portfolio Optimization and Presentation

Final Deliverables

To ensure this project lands you an interview, the UI must emphasize technical depth:

- **Live Metrics:** Real-time Ragas scores on the chat interface.
- **Architecture Diagram:** An embedded SVG showing the 11-stage pipeline.
- **Performance Case Study:** A write-up explaining why the "Winner" configuration was chosen based on data.

Recruiter Engagement Strategy

Prioritize the "System Performance" view. Most candidates show a chat box; you will show a data-driven dashboard that proves your AI is reliable.

AI Builder Prompt

"Finalize the UI by creating a 'System Health' dashboard. Use Shadcn charts to display average Faithfulness and Relevance scores over time. Include a visual representation of the RAG architecture. Add a toggle that allows users to see the 'Retrieved Chunks' and 'Evaluation Scores' behind every AI answer, making the system's inner workings fully transparent."